

Documentation technique et méthodologique des scripts du traitement automatisé d'archives numériques

Projet Nexus

Musée des Arts Décoratifs

Août 2026

Table des matières

1	Objectif	2
2	Prérequis et installation	2
2.1	Installation de WSL (Windows Subsystem for Linux)	2
2.2	Installation et configuration de VS Code	2
2.3	Connexion au serveur réseau Triarchives	2
2.4	Installation de Python et des bibliothèques nécessaires	3
3	Lancement des traitements automatiques	4
3.1	Script de décompression	4
3.2	Script de tri	5
3.3	Script de transfert et de classement	5
3.3.1	Principe de fonctionnement	6

1 Objectif

Le projet Nexus est réalisé par une étudiante du Master TNAH - Technologies Numériques Appliquées à l'Histoire de l'École nationale des Chartes. Conçu pour répondre aux besoins de préparation des données avant leur versement dans un système d'archivage électronique (SAE), il se concentre particulièrement sur les dossiers numériques d'exposition du Musée des Arts Décoratif (MAD).

L'objectif de cette documentation est de permettre à l'utilisateur (personnel des Archives) d'être autonome dans les processus de la chaîne de traitement automatisée, qui concoure au nettoyage et à la structuration des archives numériques dans l'arborescence type du musée. Le projet repose sur trois scripts réalisés à partir du langage de programmation Python.

La documentation prépare pas à pas son environnement de travail et d'exécuter les scripts.

2 Prérequis et installation

2.1 Installation de WSL (Windows Subsystem for Linux)

Pour exécuter nos scripts, nous utilisons WSL (Windows Subsystem for Linux). C'est un outil de Microsoft qui permet d'exécuter un environnement Linux directement sur Windows, sans passer par une machine virtuelle.

Pour ce faire :

- Ouvrez l'invite de commandes (PowerShell) en mode administrateur.
- Dans la fenêtre qui apparaît, tapez :

```
wsl --install
```

- Appuyez sur « Entrée », puis laissez le chargement arriver jusqu'au bout. Une fois le chargement terminé, redémarrez votre ordinateur.
- Au redémarrage, une fenêtre noire apparaît : remplissez les champs demandés (nom d'utilisateur et mot de passe Linux). Le mot de passe correspond à votre mot de passe utilisateur.

2.2 Installation et configuration de VS Code

Visual Studio Code (VS Code) est le logiciel qui permet de concevoir et de développer nos scripts afin de favoriser le traitement automatisé. Il joue le rôle de tableau de bord pour le lancement des scripts.

- Sur le site officiel (<https://code.visualstudio.com/download>), téléchargez et installez VS Code.
- À la suite du téléchargement, ouvrez VS Code.
- Cliquez sur l'icône « Extensions » dans le menu vertical à gauche.
- Tapez « WSL » dans la barre de recherche, puis cliquez sur le bouton « Installer » à côté de l'extension WSL.
- Une fois l'installation terminée, dans la liste qui apparaît en haut de l'écran, cliquez sur « Connect to WSL ».

2.3 Connexion au serveur réseau Triarchives

Pour une bonne compréhension de l'environnement de travail dans lequel vous aller travailler, il convient de comprendre le fonctionnement d'un disque réseau. Dans le cas d'espèce, il

s'identifie par la lettre (T :), c'est l'espace de test mis à disposition sur le réseau informatique de l'institution par le Service Informatique (SI) ; qui favorise l'accès à différents postes autorisés (l'archiviste et l'alternante). Il est différent d'un disque local (C :). WSL est configuré pour détecter et construire automatiquement les disques physiques internes d'un ordinateur au démarrage, sous `/mnt/c` (« mnt » représente l'emplacement où sont monter les systèmes de fichiers externes).

Dans un environnement WSL, les disques durs et serveurs réseau sont traduits automatiquement sous forme de dossiers dans `/mnt/`. Le serveur Triarchives, connecté sous Windows, devient ainsi `/mnt/t` sous Linux.

Pour accéder au serveur réseau Triarchives, plusieurs étapes sont nécessaires :

- Ouvrir le terminal, entrer `wsl`, puis la commande :

```
sudo mkdir /mnt/t
```

Cette commande crée un dossier vide nommé « t » dans le répertoire `/mnt`, qui sert de point de montage de l'espace de travail du serveur réseau.

- Un mot de passe est demandé : entrer le mot de passe de la session Linux.
- Entrer ensuite la commande :

```
sudo mount -t drvfs T: /mnt/t
```

Et entrez :

```
cd /mnt/t
```

Cette commande demande à Linux d'utiliser son pilote passerelle (`drvfs`) pour relier le contenu du lecteur réseau Windows T : au dossier `/mnt/t`.

Glossaire des commandes utilisées

- **drvfs** : système de fichiers spécial créé par Microsoft pour WSL, qui permet à Linux d'accéder aux lecteurs Windows (C :, T :, etc.) en les montant dans l'arborescence Linux. C'est ce mécanisme qui permet, par exemple, d'accéder à `C:\Users\...` depuis `/mnt/c/Users/...` dans WSL.
- **sudo** : exécuteur de commande avec droits administrateur.
- **mkdir** : commande qui sert à créer un dossier ou un répertoire.
- **mount** : commande Linux qui permet de monter un système de fichiers.

À l'issue de cette opération, la commande `ls` permet de visualiser le contenu du lecteur T : et de vérifier que le positionnement dans l'espace du serveur est correct.

Remarque : à chaque nouvelle session, il est nécessaire de répéter les commandes `wsl`, puis `sudo mount -t drvfs T: /mnt/t`, avant de se positionner dans le serveur avec `cd /mnt/t/`.

2.4 Installation de Python et des bibliothèques nécessaires

Depuis une fenêtre de terminal connectée à WSL (commande `wsl`), procéder comme suit :

- Installation du moteur Python :

```
sudo apt update && sudo apt install python3 python3-pip -y
```

— Installation des bibliothèques :

```
sudo apt install -y python3-pandas python3-openpyxl
python3-odf python3-docx python3-pypdf python3-
pdfplumber python3-pptx
```

TABLE 1 – Bibliothèques Python utilisées et leur fonction

Bibliothèque	Fonction
pandas	Lecture et manipulation des données de tableurs
openpyxl	Lire et écrire les fichiers Excel (.xlsx)
odfpy	Extraire les données des fichiers OpenDocument (.odt, .ods)
python-docx	Extraire les données des formats Word
pypdf	Lire les fichiers PDF
pdfplumber	Extraire le texte et les tableaux d'un PDF
python-pptx	Gestion des présentations PowerPoint (.pptx)
py7zr	Manipulation des archives compressées (.7z)
rarfile	Manipulation des archives compressées (.rar)

Les modules suivants, déjà intégrés à Python, sont également utilisés et ne nécessitent aucune installation :

- **os** : interaction avec le système d'exploitation, pour la gestion des dossiers et des chemins de fichiers.
- **shutil** : opérations sur les fichiers (copie, déplacement, modification, etc.).
- **unicodedata** : normalisation du texte.
- **re** : expressions régulières (regex), pour la recherche et le remplacement de motifs dans les textes.
- **logging** : système de journalisation, pour le suivi des traces des opérations effectuées.

3 Lancement des traitements automatiques

Un dossier nommé « Projet-Automatisation » est créé , à cet effet.

Après avoir sélectionné et copié votre dossier d'exposition du serveur local (C :) vers le serveur réseau (T :) (*vous pouvez le faire manuellement en glissant le dossier projet*), Rassembler dans un même emplacement : les trois scripts, le fichier de configuration, dossier racine (dossier d'exposition), le référentiel de classement.

Dans le fichier de configuration, changer les noms des dossiers : racine, du référentiel (si évolution), et l'export droid que vous allez faire à l'étape 2 du traitement(lors du tri).

Les trois scripts s'exécutent successivement : (1) décompression, (2) tri, (3) transfert et classement vers l'arborescence cible.

3.1 Script de décompression

Pour permettre à python d'extraire l'archive .7z et .rar (qui sont des bibliothèques qui se sont pas prise en compte dans le module shutil), il est nécessaire de passer par une étape préliminaire à la décompression des fichiers. Pour se faire, entrez de successivement les commandes :

```
pip install py7zr
```

Une fois la commande valider, installer le logiciel de d’archivage de compression des données 7-Zip entrez :

```
sudo apt update && sudo apt install -y p7zip-full
```

Puis :

```
sudo apt install unrar
```

Entrez la commande :

```
python3 Script_Dezip.py
```

Ce script effectue trois étapes principales :

- **Repérage de l’archive** : l’algorithme repère l’archive compressée parmi les formats pris en charge (.zip, .rar, .7z, .gz, .tar), puis affiche son chemin exact dans le dossier racine.
- **Extraction** : par la suite, l’archive repérée est décompressée et l’ensemble des fichiers qu’elle contient est extrait.
- **Renommage** : à l’issue de l’extraction, l’extension **.traite** est ajoutée au nom de l’archive d’origine.

3.2 Script de tri

Avant l’exécution, deux opérations préalables sont nécessaires :

- Scanner le dossier source à l’aide de l’outil DROID (lorsque vous faites l’export, mettre le séparateur en « ; »), qui identifie les formats et calcule l’empreinte numérique de chaque fichier.
- Créer, dans le même emplacement, un dossier nommé **A_Eliminer**.

Puis entrez :

```
python3 Tri_scores.py
```

L’algorithme s’exécute en trois étapes :

- **Repérage des dossiers et fichiers vides** : le dossier racine est scanné à la recherche de dossiers vides, à partir des données de l’export DROID.
- **Tri des données par catégories** : les répertoires du dossier source sont parcourus en profondeur afin de repérer les grandes catégories de fichiers à écarter – notamment les archives déjà traitées, les fichiers système, les fichiers vides et les doublons – puis une arborescence par catégorie est créée dans le dossier **A_Eliminer** (un choix de validation des documents dans le dossier vous sera demandée, entrer : « o » et appuyer sur la touche **Entrer** de votre clavier).
Une fois le processus terminé, l’algorithme fait un dernier tour de contrôle et identifie les dossiers/fichiers devenu vides.

3.3 Script de transfert et de classement

Créer dans l’emplacement prévu ci-dessus, un dossier, que vous appellerez : **Arborescence** (vous pouvez le renommer comme vous le souhaitez, ceci est juste un exemple). Puis, dans le fichier de configuration, repérer la partie dédiée à l’arborescence et entrez-y le nom que lui avez attribué.

```
python3 Script_Arborescence.py
```

L’algorithme crée une arborescence conforme au plan de classement, avec l’ensemble de ses composantes.

3.3.1 Principe de fonctionnement

Ce script constitue le moteur de classement automatique des fichiers en vrac : il range les documents dans une arborescence de dossiers structurée, conformément au plan de classement du MAD.

1. **Lecture du référentiel Excel** – le script lit le fichier Excel décrivant l'arborescence cible (dossiers 01_..., 01_01_..., etc.) ainsi que les mots-clés associés à chaque dossier. Il construit un dictionnaire de correspondance (`mapping_ejection`) associant chaque mot-clé à un chemin de destination.
2. **Conversion des chemins Windows vers WSL** – la fonction `Conversion_chemin` transforme les chemins Windows (`C:\dossier`) en chemins compatibles WSL/Linux (`/mnt/d/dossier`).
3. **Copie sécurisée des fichiers** – Le script fait une copie des fichiers en conservant les métadonnées (`Copyfile`), telles que : l'auteur, la date d'origine de création, les dates de modification.
4. **Extraction de texte multi-formats** – Il extrait et analyse le contenu textuel des fichiers `.docx`, `.pptx`, `.ods`, `.pdf` (via `pdfplumber`, puis `pypdf` en repli), `.txt` et `.csv`.
5. **Moteur de classement en deux phases** :
 - *Phase 1 (tri de surface)* : pour chaque fichier, plusieurs critères sont testés successivement – correspondance stricte dans le titre du fichier, puis héritage du dossier parent si celui-ci correspond à un mot-clé, code photothèque connu (préfixe type inventaire), .
 - *Phase 2 (analyse sémantique)* : pour les fichiers non classés en phase 1, le texte du contenu est extrait et un score par catégorie est calculé (`calculer_score_categorie`), en croisant mots-clés, synonymes métiers (`config.SYNONYMES_METIERS`) et position dans le titre ou dans le contenu. Le fichier est affecté à la catégorie obtenant le score le plus élevé. En dernier recours, un repli s'opère vers la catégorie `í` iconographie `z` pour les images, ou vers un dossier portant le même nom que le dossier parent d'origine.

Il s'agit d'un système de classement heuristique en cascade – nom de fichier exact, puis code d'inventaire, puis dossier parent, puis analyse sémantique du contenu, puis repli générique – qui vise à reconstituer automatiquement une arborescence documentaire cohérente à partir d'un vrac de fichiers hétérogènes, sans intervention manuelle systématique.

En résumé Il convient de préciser qu'un système de *journalisation* – permettant d'apporter une vue sur les différentes opérations de différents scripts. Un dossier log est créé automatiquement. Il convient d'adapter le vocabulaire métier dans le point config propre à chaque exposition.